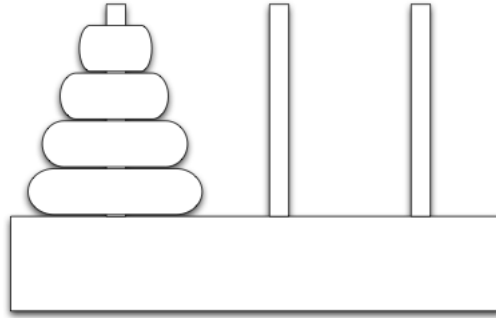


Section 1: Search

1 Towers of Hanoi



The Towers of Hanoi is a famous problem for studying recursion in computer science and recurrence equations in discrete mathematics. We start with N discs of varying sizes on a peg (stacked in order according to size), and two empty pegs. We are allowed to move a disc from one peg to another, but we are never allowed to move a larger disc on top of a smaller disc. The goal is to move all the discs to the rightmost peg (see figure).

In this problem, we will formulate the Towers of Hanoi as a search problem.

- (a) Propose a state representation for the problem

One possible state representation would be to store three lists, corresponding to which discs are on which peg. If we assume that the N discs are numbered in order of increasing size $1, \dots, n$, then we can represent each peg as an ordered list of integers corresponding to which discs are on that peg.

- (b) What is the size of the state space? If there are k pegs and n disks, then the size of the state space is k^n . For this setup, the size is 3^N .

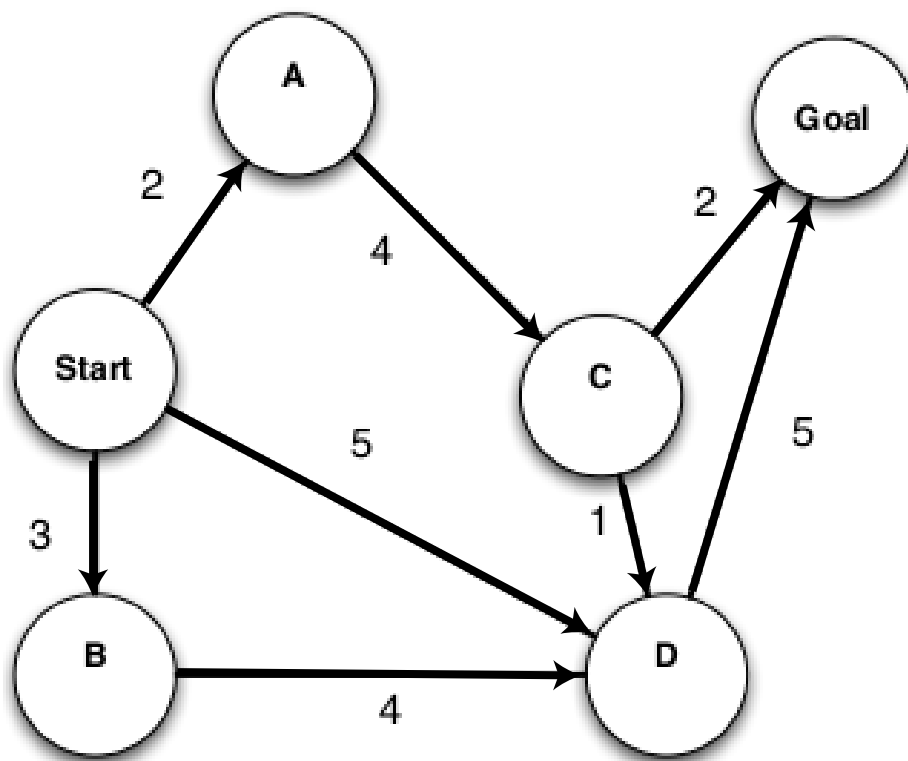
- (c) What is the start state? $([1, \dots, n], [], [])$

- (d) From a given state, what actions are legal?

We can pop the first integer from any list (i.e., peg) and push it onto the front of another list (peg), so long as it is smaller than the integer currently at the front of the list being pushed to (i.e., peg being moved to).

- (e) What is the goal test? Is the state the same as $([], [], [1, \dots, n])$?

2 Search algorithms in action



For each of the following graph search strategies, work out the order in which states are expanded, as well as the path returned by graph search. In all cases, assume ties resolve in such a way that states with earlier alphabetical order are expanded first. Remember that in graph search, a state is expanded only once.

a) Depth-first search.

States Expanded: Start, A, C, D, Goal

Path Returned: Start-A-C-D-Goal

b) Breadth-first search.

States Expanded: Start, A, B, D, C, Goal

Path Returned: Start-D-Goal

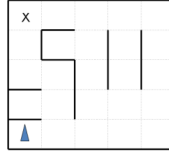
c) Uniform cost search.

States Expanded: Start, A, B, D, C, Goal

Path Returned: Start-A-C-Goal

3 Search and Heuristics

Imagine a car-like agent wishes to exit a maze like the one shown below:



The agent is directional and at all times faces some direction $d \in (N, S, E, W)$. With a single action, the agent can *either* move forward at an adjustable velocity v *or* turn. The turning actions are *left* and *right*, which change the agent's direction by 90 degrees. Turning is only permitted when the velocity is zero (and leaves it at zero). The moving actions are *fast* and *slow*. *Fast* increments the velocity by 1 and *slow* decrements the velocity by 1; in both cases the agent then moves a number of squares equal to its NEW adjusted velocity (see example below). A consequence of this formulation is that it is impossible for the agent to move in the same nonzero velocity for two consecutive timesteps. Any action that would result in a collision with a wall crashes the agent and is illegal. Any action that would reduce v below 0 or above a maximum speed $V_{\max}$ is also illegal. The agent's goal is to find a plan which parks it (stationary) on the exit square using as few actions (time steps) as possible.

As an example: if at timestep t the agent's current velocity is 2, by taking the *fast* action, the agent first increases the velocity to 3 and move 3 squares forward, such that at timestep $t + 1$ the agent's current velocity will be 3 and will be 3 squares away from where it was at timestep t . If instead the agent takes the *slow* action, it first decreases its velocity to 1 and then moves 1 square forward, such that at timestep $t + 1$ the agent's current velocity will be 1 and will be 1 squares away from where it was at timestep t . If, with an instantaneous velocity of 1 at timestep $t + 1$, it takes the *slow* action again, the agent's current velocity will become 0, and it will not move at timestep $t + 1$. Then at timestep $t + 2$, it will be free to turn if it wishes. Note that the agent could not have turned at timestep $t + 1$ when it had a current velocity of 1, because it has to be stationary to turn.

1. If the grid is M by N , what is the size of the state space? Justify your answer. You should assume that all configurations are reachable from the start state.

The size of the state space is $4MN(V_{\max} + 1)$. The state representation is (direction facing, x, y , speed). Note that the speed can take any value in $\{0, \dots, V_{\max}\}$.

2. Is the Manhattan distance from the agent's location to the exit's location admissible? Why or why not?

No, Manhattan distance is not an admissible heuristic. The agent can move at an average speed of greater than 1 (by first speeding up to $V_{\max}$ and then slowing down to 0 as it reaches the goal), and so can reach the goal in less time steps than there are squares between it and the goal. A specific example: A timestep 0, the agent's starts stationary at square 0 and the target is 9 squares away at square 9. At timestep 0, the agent takes the *fast* action and ends up at square 1 with a velocity of 1. At timestep 1, the agent takes the *fast* action and ends up at square 3 with a velocity of 2. At timestep 2, the agent takes the *fast* action and ends up at square 6 with a velocity of 3. At timestep 3, the agent takes the *slow* action and ends up at square 8 with a velocity of 2. At timestep 4, the agent takes the *slow* action and ends up at square 9 with a velocity of 1. At timestep 5, the agent takes the *slow* action and stays at square 9 with a velocity of 0. Therefore, the agent can move 9 squares by taking 6 actions.

3. State and justify a non-trivial admissible heuristic for this problem which is not the Manhattan distance to the exit.

There are many answers to this question. Here are a few, in order of weakest to strongest:

- (a) The number of turns required for the agent to face the goal.
- (b) Consider a relaxation of the problem where there are no walls, the agent can turn, change speed arbitrarily, and maintain constant velocity. In this relaxed problem, the agent would move with V_{max} , and then suddenly stop at the goal, thus taking $d_{manhattan}/V_{max}$ time.
- (c) We can improve the above relaxation by accounting for the acceleration and deceleration dynamics. In this case the agent will have to accelerate from 0 from the start state, maintain a constant velocity of V_{max} , and slow down to 0 when it is about to reach the goal. Note that this heuristic will always return a greater value than the previous one, but is still not an overestimate of the true cost to reach the goal. We can say that this heuristic *dominates* the previous one.

In particular, let us assume that $d_{manhattan}$ is greater than and equal to the distance it takes to accelerate to and decelerate from V_{max} (In the case that $d_{manhattan}$ is smaller than this distance, we can still use $d_{manhattan}/V_{max}$ as a heuristic). We can break up the $d_{manhattan}$ into three parts: d_{accel} , $d_{V_{max}}$, and d_{decel} . The agent travels a distance of d_{accel} when it accelerates from 0 to V_{max} velocity. The agent travels a distance of d_{decel} when it decelerates from V_{max} to 0 velocity. In between acceleration and deceleration, the agent travels a distance of $d_{V_{max}} = d_{manhattan} - d_{accel} - d_{decel}$. $d_{accel} = 1 + 2 + 3 + V_{max} = \frac{(V_{max})(V_{max}+1)}{2}$ and $d_{decel} = (V_{max} - 1) + (V_{max} - 2) + \dots + 1 + 0 = \frac{(V_{max})(V_{max}-1)}{2}$. So $d_{V_{max}} = d_{manhattan} - \frac{(V_{max})(V_{max}+1)}{2} - \frac{(V_{max})(V_{max}-1)}{2} = d_{manhattan} - V_{max}^2$. It takes V_{max} steps to travel the initial d_{accel} , $\frac{d_{manhattan} - V_{max}^2}{V_{max}}$ steps to travel the middle $d_{V_{max}}$ and V_{max} steps to travel the last d_{decel} . Therefore, our heuristic is

$$\begin{cases} \frac{d_{manhattan}}{V_{max}}, & \text{if } d_{manhattan} \leq V_{max}^2 = d_{accel} + d_{decel} \\ \frac{d_{manhattan}}{V_{max}} + V_{max}, & \text{if } d_{manhattan} > V_{max}^2 = d_{accel} + d_{decel} \end{cases}$$

4. If we used an inadmissible heuristic in A* graph search, would the search be complete? Would it be optimal?

If the heuristic function is bounded, then A* graph search would visit all the nodes eventually, and would find a path to the goal state if there exists one. An inadmissible heuristic does not guarantee optimality as it can make the good optimal goal look as though it is very far off, and take you to a suboptimal goal.

5. If we used an *admissible* heuristic in A* graph search, is it guaranteed to return an optimal solution? What if the heuristic was consistent? What if we were using A* tree search instead of A* graph search?

Although admissible heuristics guarantee optimality for A* *tree* search, they do not necessarily guarantee optimality for A* *graph* search; they are only guaranteed to return an optimal solution if they are consistent as well.

6. Give a general advantage that an inadmissible heuristic might have over an admissible one.

The time to solve an A* search problem is a function of two factors: the number of nodes expanded, and the time spent per node. An inadmissible heuristic may be faster to compute, leading to a solution that is obtained faster due to less time spent per node. It can also be a closer estimate to the actual cost function (even though at times it will overestimate!), thus expanding less nodes. We lose the guarantee of optimality by using an inadmissible heuristic. But sometimes we may be okay with finding a suboptimal solution to a search problem.

Section 2 Solutions

CSPs

CSPs are defined by three factors:

1. *Variables* - CSPs possess a set of N variables $X_1, \dots, X_N$ that can each take on a single value from some defined set of values.
2. *Domain* - A set $\{x_1, \dots, x_d\}$ representing all possible values that a CSP variable can take on.
3. *Constraints* - Constraints define restrictions on the values of variables, potentially with regard to other variables.

CSPs are often represented as constraint graphs, where nodes represent variables and edges represent constraints between them.

- *Unary Constraints* - Unary constraints involve a single variable in the CSP. They are not represented in constraint graphs, instead simply being used to prune the domain of the variable they constrain when necessary.
- *Binary Constraints* - Binary constraints involve two variables. They're represented in constraint graphs as traditional graph edges.
- *Higher-order Constraints* - Constraints involving three or more variables can also be represented with edges in a CSP graph.

In **forward checking**, whenever a value is assigned to a variable X_i , forward checking prunes the domains of unassigned variables that share a constraint with X_i that would violate the constraint if assigned. The idea of forward checking can be generalized into the principle of **arc consistency**. For arc consistency, we interpret each undirected edge of the constraint graph for a CSP as two directed edges pointing in opposite directions. Each of these directed edges is called an **arc**. The arc consistency algorithm works as follows:

- Begin by storing all arcs in the constraint graph for the CSP in a queue Q .
- Iteratively remove arcs from Q and enforce the condition that in each removed arc $X_i \rightarrow X_j$, for every remaining value v for the tail variable X_i , there is at least one remaining value w for the head variable X_j such that $X_i = v, X_j = w$ does not violate any constraints. If some value v for X_i would not work with any of the remaining values for X_j , we remove v from the set of possible values for X_i .
- If at least one value is removed for X_i when enforcing arc consistency for an arc $X_i \rightarrow X_j$, add arcs of the form $X_k \rightarrow X_i$ to Q , for all unassigned variables X_k . If an arc $X_k \rightarrow X_i$ is already in Q during this step, it doesn't need to be added again.
- Continue until Q is empty, or the domain of some variable is empty and triggers a backtrack.

We've delineated that when solving a CSP, we fix some ordering for both the variables and values involved. In practice, it's often much more effective to compute the next variable and corresponding value "on the fly" with two broad principles, **minimum remaining values** and **least constraining value**:

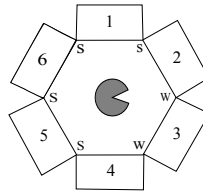
- *Minimum Remaining Values (MRV)* - When selecting which variable to assign next, using an MRV policy chooses whichever unassigned variable has the fewest valid remaining values (the *most constrained variable*).
- *Least Constraining Value (LCV)* - Similarly, when selecting which value to assign next, a good policy to implement is to select the value that prunes the fewest values from the domains of the remaining unassigned values.

1 CSPs: Trapped Pacman

Pacman is trapped! He is surrounded by mysterious corridors, each of which leads to either a pit (P), a ghost (G), or an exit (E). In order to escape, he needs to figure out which corridors, if any, lead to an exit and freedom, rather than the certain doom of a pit or a ghost.

The one sign of what lies behind the corridors is the wind: a pit produces a strong breeze (S) and an exit produces a weak breeze (W), while a ghost doesn't produce any breeze at all. Unfortunately, Pacman cannot measure the strength of the breeze at a specific corridor. Instead, he can stand *between* two adjacent corridors and feel the max of the two breezes. For example, if he stands between a pit and an exit he will sense a strong (S) breeze, while if he stands between an exit and a ghost, he will sense a weak (W) breeze. The measurements for all intersections are shown in the figure below.

Also, while the total number of exits might be zero, one, or more, Pacman knows that two neighboring squares will *not* both be exits.



Pacman models this problem using variables X_i for each corridor i and domains P, G, and E.

- (a) State the binary and/or unary constraints for this CSP (either implicitly or explicitly).

Binary:

$$\begin{aligned} X_1 = P \text{ or } X_2 = P, & \quad X_2 = E \text{ or } X_3 = E, \\ X_3 = E \text{ or } X_4 = E, & \quad X_4 = P \text{ or } X_5 = P, \\ X_5 = P \text{ or } X_6 = P, & \quad X_1 = P \text{ or } X_6 = P, \\ \forall i, j \text{ s.t. Adj}(i, j) & \neg (X_i = E \text{ and } X_j = E) \end{aligned}$$

Unary:

$$\begin{aligned} X_2 &\neq P, \\ X_3 &\neq P, \\ X_4 &\neq P \end{aligned}$$

Note: This is just one of many solutions. The answers below will be based on this formulation.

- (b) Suppose we assign X_1 to E. Perform forward checking after this assignment. Also, enforce unary constraints.

X_1			E
X_2			
X_3		G	E
X_4		G	E
X_5	P	G	E
X_6	P		

(c) Suppose forward checking returns the following set of possible assignments:

X_1	P		
X_2		G	E
X_3		G	E
X_4		G	E
X_5	P		
X_6	P	G	E

According to MRV, which variable or variables could the solver assign first?

X_1 or X_5 (tie breaking)

(d) Assume that Pacman knows that $X_6 = G$. List all the solutions of this CSP or write *none* if no solutions exist.

(P,E,G,E,P,G)
(P,G,E,G,P,G)

2 CSP: Air Traffic Control

We have five planes: A, B, C, D, and E and two runways: international and domestic. We would like to schedule a time slot and runway for each aircraft to **either** land or take off. We have four time slots: $\{1, 2, 3, 4\}$ for each runway, during which we can schedule a landing or take off of a plane. We must find an assignment that meets the following constraints:

- Plane B has lost an engine and must land in time slot 1.
 - Plane D can only arrive at the airport to land during or after time slot 3.
 - Plane A is running low on fuel but can last until at most time slot 2.
 - Plane D must land before plane C takes off, because some passengers must transfer from D to C.
 - No two aircrafts can reserve the same time slot for the same runway.
- (a) Complete the formulation of this problem as a CSP in terms of variables, domains, and constraints (both unary and binary). Constraints should be expressed implicitly using mathematical or logical notation rather than with words.

Variables: A, B, C, D, E for each plane.

Domains: a tuple *(runway type, time slot)* for runway type $\in \{international, domestic\}$ and time slot $\in \{1, 2, 3, 4\}$.

Constraints:

$$\begin{array}{ll}
 B[1] = 1 & A[1] \leq 2 \\
 D[1] \geq 3 & D[1] < C[1] \\
 & A \neq B \neq C \neq D \neq E
 \end{array}$$

- (b) For the following subparts, we add the following two constraints:

- Planes A, B, and C cater to international flights and can only use the international runway.
- Planes D and E cater to domestic flights and can only use the domestic runway.

- (i) With the addition of the two constraints above, we completely reformulate the CSP. You are given the variables and domains of the new formulation. Complete the constraint graph for this problem given the original constraints and the two added ones.

Variables: A, B, C, D, E for each plane.

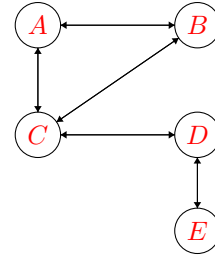
the planes that use the same runways.

Domains: $\{1, 2, 3, 4\}$

Explanation of Constraint Graph:

We can now encode the runway information into the identity of the variable, since each runway has more than enough time slots for the planes it serves. We represent the non-colliding time slot constraint as a binary constraint between

Constraint Graph:



- (ii) What are the domains of the variables after enforcing arc-consistency? Begin by enforcing unary constraints. (Cross out values that are no longer in the domain.)

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

- (iii) Arc-consistency can be rather expensive to enforce, and we believe that we can obtain faster solutions using only **forward-checking** on our variable assignments. Using the Minimum Remaining Values heuristic, perform backtracking search on the graph, breaking ties by picking lower values and characters first. List the $(variable, assignment)$ pairs in the order they occur (including the assignments that are reverted upon reaching a dead end). Enforce unary constraints before starting the search.

(You don't have to use this table, it won't be graded.)

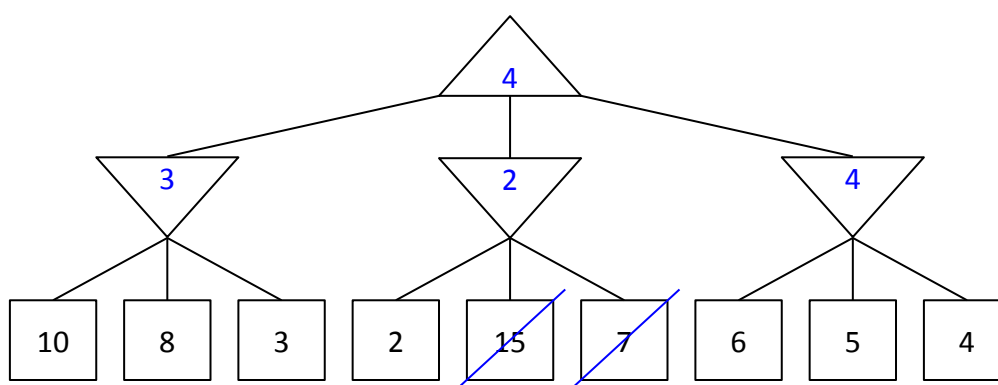
A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Answer: (B, 1), (A, 2), (C, 3), (C, 4), (D, 3), (E, 1)

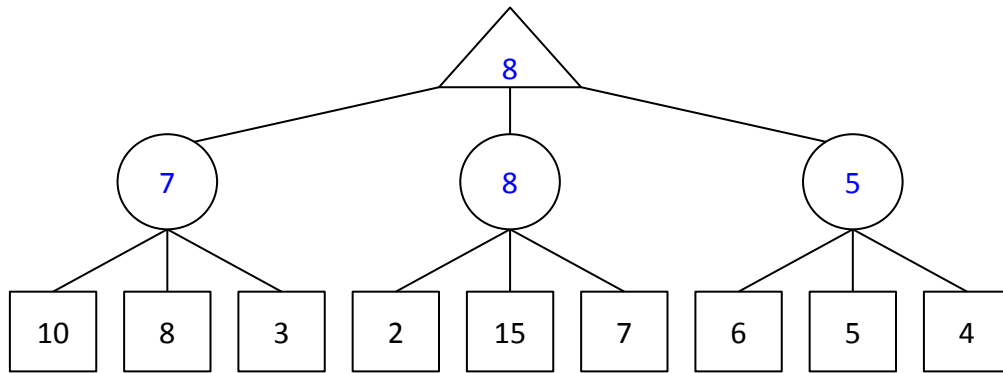
Section 3 Solutions

1 Games

- (a) Consider the zero-sum game tree shown below. Triangles that point up, such as at the top node (root), represent choices for the maximizing player; triangles that point down represent choices for the minimizing player. Assuming both players act optimally, fill in the minimax value of each node.



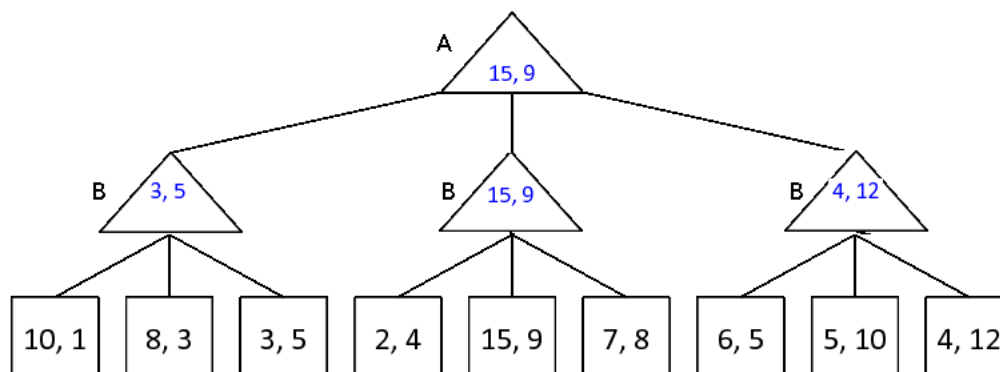
- (b) Which nodes can be pruned from the game tree above through alpha-beta pruning? If no nodes can be pruned, explain why not. Assume the search goes from left to right; when choosing which child to visit first, choose the left-most unvisited child.
- (c) Again, consider the same zero-sum game tree, except that now, instead of a minimizing player, we have a chance node that will select one of the three values uniformly at random. Fill in the expectimax value of each node. The game tree is redrawn below for your convenience.



- (d) Which nodes can be pruned from the game tree above through alpha-beta pruning? If no nodes can be pruned, explain why not. **No nodes can be pruned.** There will always be the possibility that an as-yet-unvisited leaf of the current parent chance node will have a very high value, which increases the overall average value for that chance node. For example, when we see that leaf 4 has a value of 2, which is much less than the value of the left chance node, 7, at this point we cannot make any assumptions about how the value of the middle chance node will ultimately be more or less in value than the left chance node. As it turns out, the leaf 5 has a value of 15, which brings the expected value of the middle chance node to 8, which is greater than the value of the left chance node. In the case where there is an upper bound to the value of a leaf node, there is a possibility of pruning: suppose that an upper bound of +10 applies only to the children of the rightmost chance node. In this case, after seeing that leaf 7 has a value of 6 and leaf 8 has a value of 5, the best possible value that the rightmost chance node can take on is $\frac{6+5+10}{3} = 7$, which is less than 8, the value of the middle chance node. Therefore, it is possible to prune leaf 9 in this case.

2 Nonzero-sum Games

1. Let's look at a non-zero-sum version of a game. In this formulation, player A's utility will be represented as the first of the two leaf numbers, and player B's utility will be represented as the second of the two leaf numbers. Fill in this non-zero game tree assuming each player is acting optimally.



2. Which nodes can be pruned from the game tree above through alpha-beta pruning? If no nodes can be pruned, explain why not. **No nodes can be pruned. Because this game is non-zero-sum, there can exist a leaf node anywhere in the tree that is good for both player A and player B.**